Guia de Revisão Intensiva: Estrutura de Dados

Módulos I, II, III e IV — Resumo Teórico & Prático para Prova

1. MEMÓRIA, PONTEIROS & ALOCAÇÃO DINÂMICA

Ponteiros e Operadores

Variável que armazena o **endereço de memória** de outra variável.

- `&` (Endereço): Retorna a posição de memória.
- `*` (Desreferenciação): Acessa o valor no endereço.
- `int *p = &x;` (`p` guarda endereço, `*p` guarda valor).

Alocação de Memória

- **Estática:** Definida na compilação. Tamanho fixo na memória Stack. Ex: `int v[100];`
- **Dinâmica:** Definida na execução na Heap. Permite redimensionar. Ex: `malloc()`, `free()`.
- **Memory Leak:** Ocorre quando se aloca com `malloc` sem liberar com `free`.

Tipos de Dados: Homogêneos vs. Heterogêneos

Homogêneos (Arrays/Vetores): Sequência contígua de elementos do *mesmo tipo*. Acesso direto via índice `O(1)`.

Heterogêneos (Structs/Registros): Agrupamento de variáveis de *tipos diferentes* sob um mesmo nome de bloco.

2. ESTRUTURAS LINEARES BÁSICAS

Listas Encadeadas

Nós espalhados na memória conectados por ponteiros.

- **Simples:** Ponteiro para o próximo nó.
- **Dupla:** Ponteiros para próximo e anterior.
- **Circular:** Último nó aponta para o primeiro.

Pilhas (Stacks)

Regra **LIFO** (Last In, First Out — Último a entrar, primeiro a sair).

- `Push()` : Insere no topo.
- `Pop()` : Remove do topo.
- **Aplicações:** Call stack, Desfazer (Ctrl+Z).

Filas (Queues)

Regra **FIFO** (First In, First Out — Primeiro a entrar, primeiro a sair).

- `Enqueue()` : Insere no fim.
- `Dequeue()` : Remove do início.
- **Aplicações:** Impressão, Buffers.



3. MÉTODOS DE BUSCA & ALGORITMOS DE ORDENAÇÃO

Busca Sequencial (Linear)

Percorre o vetor elemento por elemento a partir do início.

- **Requisito:** Funciona em vetores ordenados ou não.
- **Complexidade:** `O(n)` em todos os casos médios/piores.

Busca Binária

Divide o espaço de busca ao meio repetidamente.

- **Requisito:** O vetor deve estar **OBRIGATORIAMENTE ORDENADO**.
- **Complexidade:** `O(log n)`. Muito eficiente.

Tabela Comparativa de Algoritmos de Ordenação (Sorting)

ALGORITMO	MELHOR CASO	CASO MÉDIO	PIOR CASO	ESTÁVEL?	DESCRIÇÃO / MECANISMO PRINCIPAL
Bubble Sort	`O(n)`	`O(n²)`	`O(n²)`	Sim	Compara elementos adjacentes e troca os fora de ordem. "Flutua" o maior.
Selection Sort	`O(n²)`	`O(n²)`	`O(n²)`	Não	Procura o menor elemento do vetor e o coloca na primeira posição livre.
Insertion Sort	`O(n)`	`O(n²)`	`O(n²)`	Sim	Insere cada elemento na posição correta da sublista já ordenada (cartas).
Merge Sort	`O(n log n)`	`O(n log n)`	`O(n log n)`	Sim	Dividir e Conquistar: divide ao meio, ordena as partes e combina (merge).
Quick Sort	`O(n log n)`	`O(n log n)`	`O(n²)`	Não	Escolhe um Pivô, particiona (menores à esquerda, maiores à direita) e recursiona.
Heap Sort	`O(n log n)`	`O(n log n)`	`O(n log n)`	Não	Monta uma estrutura de Max-Heap e remove o maior elemento sucessivamente.

Detalhes do QuickSort

• **Pivô:** Elemento de referência usado para dividir o vetor.

• **Particionamento:** Coloca menores à esquerda e maiores à direita do pivô.

• **Pior Caso $O(n^2)$:** Ocorre quando o pivô escolhido é sempre o menor ou maior elemento (ex: vetor já ordenado com pivô na ponta).

Detalhes do MergeSort

• **Recursão:** Divide o vetor em subvetores de tamanho 1.

• **Intercalação (Merge):** Junta dois vetores ordenados em um terceiro vetor de forma eficiente.

• **Desvantagem:** Consome memória auxiliar proporcional a $O(n)$.

Dica de Prova: Algoritmo *Estável* é aquele que mantém a ordem relativa de elementos com chaves iguais. **Insertion** e **Merge** são estáveis; **Quick** e **Heap** não são.

4. ÁRVORES BINÁRIAS DE BUSCA (BST) & ÁRVORES AVL

Terminologia de Árvores

- **Raiz:** Nó do topo, sem pai.
- **Folha:** Nó sem nenhum filho.
- **Grau de um Nó:** Número de filhos diretos.
- **Altura (h):** Maior caminho da raiz até uma folha.

Regra da BST (Binary Search Tree)

Para qualquer nó X da árvore:

- Valores na **Subárvore Esquerda** < X.
- Valores na **Subárvore Direita** > X.
- Garante busca em `O(log n)` se balanceada.

Percursos em Árvores Binárias (Traversals)

1. Pré-Ordem

`Raiz → Esquerda → Direita`

Visita a raiz primeiro. Usado para clonar ou exportar a estrutura da árvore.

2. Em-Ordem (In-Order)

`Esquerda → Raiz → Direita`

Visita os nós em **ordem estritamente crescente** em uma BST!

3. Pós-Ordem

`Esquerda → Direita → Raiz`

Visita a raiz por último. Usado para deletar nós da memória ou avaliar expressões.

```
Exemplo de Árvore BST: Percursos para esta Árvore: (8) • Pré-Ordem: 8 -> 3 -> 1 -> 6 -> 10 / \ • E
```

5. BALANCEAMENTO — ÁRVORE AVL

Se inserirmos elementos ordenados numa BST comum, ela vira uma "lista" com busca `O(n)`. A **Árvore AVL** é uma BST auto-balanceada que evita esse problema.

Fator de Balanceamento (FB): $FB = \text{altura}(\text{subárvore esquerda}) - \text{altura}(\text{subárvore direita})$
Regra da AVL: Em TODOS os nós, o fator de balanceamento deve ser obrigatoriamente **-1, 0 ou $+1$**. Se $FB = \pm 2$, a árvore deve ser rebalanceada.

Tipos de Rotação na Árvore AVL

Rotações Simples (Linha Reta)

• **Rotação à Direita (LL):** Usada quando o desbalanceamento ocorre na subárvore esquerda do filho esquerdo ($FB = +2$).

• **Rotação à Esquerda (RR):** Usada quando o desbalanceamento ocorre na subárvore direita do filho direito ($FB = -2$).

Rotações Duplas (Zig-Zag)

• **Dupla Esquerda-Direita (LR):** Desbalanceamento no filho direito do nó esquerdo. (Rotação simples à esq. no filho + simples à dir. no nó).

• **Dupla Direita-Esquerda (RL):** Desbalanceamento no filho esquerdo do nó direito.

6. TABELAS HASH (ESPALHAMENTO)

Estrutura para busca, inserção e remoção em tempo constante médio: `O(1)`.

• **Função Hash:** Converte uma chave num numérica no índice da tabela. Ex: $h(k) = k \bmod \text{tamanho_tabela}$.

• **Colisão:** Quando duas chaves diferentes geram o mesmo índice na tabela.

Tratamento: Encadeamento Exterior

Cada posição do vetor da tabela guarda uma **Lista Encadeada**. Quando ocorre colisão, o elemento é adicionado no fim da lista daquela posição.

Tratamento: Endereçamento Aberto

Todos os elementos ficam no próprio vetor. Se colidir, procura outra vaga:

- **Sondagem Linear:** Testa a vaga seguinte $i+1, i+2$.
- **Sondagem Quadrática:** Saltos com $i + 1^2, i + 2^2$.
- **Hash Duplo:** Usa uma 2ª função hash para definir o salto.

7. GRAFOS & ALGORITMOS DE BUSCA

Estrutura $G = (V, E)$ composta por um conjunto de **Vértices (V)** e **Arestas (E)**.

Representação de Grafos

• **Matriz de Adjacência:** Matriz $V \times V$. Consome memória $O(V^2)$. Ideal para grafos densos.

• **Lista de Adjacência:** Vetor de listas. Consome $O(V + E)$. Ideal para grafos esparsos.

Algoritmo de Dijkstra

Encontra o **caminho mínimo** de um nó de origem para todos os outros em grafos com pesos *não-negativos*. Usa fila de prioridade.

Busca em Largura (BFS)

• Explora em nível/camadas por distância.

• Utiliza uma **FILA (Queue)**.

• Encontra caminho mais curto em número de arestas.

Busca em Profundidade (DFS)

• Explora um caminho o mais fundo possível (*backtracking*).

• Utiliza uma **PILHA (Stack)** ou Recursão.

• Usado para detecção de ciclos e ordenação topológica.

8. RESUMO MESTRE DE COMPLEXIDADES BIG-O

ESTRUTURA / ALGORITMO	ACESSO / BUSCA	INSERÇÃO	REMOÇÃO
Vetor (Array)	Acesso: `O(1)` / Busca: `O(n)`	`O(n)` (deslocamento)	`O(n)`
Lista Encadeada	`O(n)` (percurso)	`O(1)` (no nó)	`O(1)` (no nó)
Pilha (Stack) / Fila (Queue)	Apenas Topo / Início	`O(1)`	`O(1)`
Árvore BST (Média / Pior)	`O(log n)` / `O(n)`	`O(log n)` / `O(n)`	`O(log n)` / `O(n)`
Árvore AVL (Garantido)	`O(log n)`	`O(log n)`	`O(log n)`
Tabela Hash (Média / Pior)	`O(1)` / `O(n)`	`O(1)` / `O(n)`	`O(1)` / `O(n)`